Proyecto Final - Arquitectura Orientada a Microservicios

Contexto del Negocio

Su equipo ha sido contratado para diseñar y desarrollar un **sistema de gestión de empleados** para una empresa en crecimiento. El departamento de **Recursos Humanos (RRHH)** necesita automatizar y modernizar sus procesos críticos relacionados con el ciclo de vida del empleado, desde su incorporación hasta su salida de la organización.

Actualmente, todos estos procesos se realizan manualmente, lo que genera:

- **Retrasos** en la creación de credenciales para nuevos empleados
- **Inconsistencias** en la gestión de accesos
- **Falta de visibilidad** sobre el estado de las cuentas
- **Problemas de seguridad** por cuentas activas de empleados que ya no laboran en la empresa
- **Dificultad** para notificar cambios importantes a los empleados

La empresa requiere una solución moderna, escalable y mantenible basada en arquitectura de microservicios que permita automatizar estos procesos críticos.

Descripción del Sistema

Deberá construir un sistema con **arquitectura basada en microservicios** que soporte el siguiente ciclo de vida del empleado:

1. Onboarding (Incorporación)

Cuando RRHH registra un nuevo empleado en el sistema:

- Se debe crear automáticamente la cuenta de acceso del empleado, **en estado inactivo y sin contraseña válida**
- Se debe generar un **token de activación con expiración** y enviarlo al empleado por correo electrónico, para que él mismo establezca su contraseña

- El empleado debe poder activar su cuenta, acceder al sistema y gestionar su perfil personal

> ⚠ **Caution**
>
> **El sistema nunca genera ni transmite contraseñas.** No se envían contraseñas temporales por
> correo, ni en texto plano ni de ninguna otra forma. La contraseña la establece únicamente el empleado, usando el token de activación. Las contraseñas se almacenan siempre hasheadas.

2. Gestión de Perfil

Los empleados autenticados deben poder:

- Actualizar información personal (nombre, apellido, teléfono)
- Actualizar información de contacto (dirección, ciudad, código postal)
- Actualizar información profesional (cargo, área, número de empleado)
- Ver su información de manera centralizada

3. Gestión de Vacaciones

Cuando RRHH programa vacaciones para un empleado:

- La cuenta del empleado se desactiva automáticamente en la fecha de inicio de las vacaciones
- La cuenta se reactiva automáticamente cuando el empleado regresa (fecha de fin de vacaciones)
- Se envía una notificación al empleado confirmando el período de vacaciones

4. Offboarding (Salida)

Cuando un empleado es retirado de la empresa:

- RRHH marca al empleado como "inactivo" o "retirado"
- Todas las credenciales y accesos del empleado se desactivan permanentemente
- Se genera un registro de auditoría de la fecha y hora de la desactivación

5. Observabilidad

El sistema debe proveer mecanismos de observabilidad que permitan:

- **Logs centralizados**: Todos los microservicios deben enviar logs estructurados a un sistema central

- **Monitoreo**: Visualización del estado y salud de cada microservicio
- **Trazabilidad distribuida**: Seguir una petición a través de todos los microservicios que intervienen en ella, sin importar el lenguaje en que estén escritos
- **Alertas**: Notificaciones cuando ocurran errores críticos o comportamientos anómalos

Microservicios Requeridos

Su solución debe incluir al menos los siguientes microservicios (pueden ser propios o de terceros):

Microservicios Principales

1. Microservicio de Gestión de Empleados

- CRUD completo de empleados (crear, leer, actualizar, retirar)
- Estados del empleado: `ACTIVO`, `EN_VACACIONES`, `RETIRADO`
- El retiro es una **transición de estado, no un borrado**: el registro se conserva con su `fechaRetiro` para auditoría
- Consulta de auditoría: listar empleados `RETIRADO` con filtro por rango de fechas
- Validación de unicidad de `email` y `numeroEmpleado`
- Publicar eventos cuando se crea, actualiza o retira un empleado
- **Campos obligatorios**: `id`, `nombre`, `apellido`, `email`, `numeroEmpleado`, `cargo`, `area`, `departamentoId`, `fechaIngreso`, `estado`

2. Microservicio de Gestión de Departamentos

- CRUD de departamentos (`id`, `nombre`, `descripcion`)
- Es la fuente de verdad de los departamentos de la organización
- El microservicio de Empleados **valida contra este servicio** que el `departamentoId` exista al registrar un empleado; si no existe, la operación se rechaza con `400 Bad Request`
- Es el punto de comunicación **sincrónica** entre microservicios del ecosistema, y por lo tanto donde deben aplicarse los patrones de resiliencia (timeout, reintentos, circuit breaker con fallback)

3. Microservicio de Autenticación y Autorización

Puede resolverse de **dos formas, ambas válidas**:

(a) Servicio propio — emite y valida tokens JWT con secreto compartido.
(b) Identity Provider — Keycloak, Authentik u otro IdP OAuth2/OIDC emite los tokens; el API Gateway los valida por **JWKS** (sin secreto compartido) y se conserva un servicio reducido que sincroniza el ciclo de vida de la cuenta con los eventos de empleado.

> La opción **(b)** es la solución de nivel productivo y **otorga puntaje adicional** en el criterio de *Seguridad y Configuración*.

En cualquiera de las dos formas, el componente debe:

- Crear la cuenta automáticamente al consumir `empleado.creado`, en estado inactivo y sin contraseña válida, generando un **token de activación con expiración**
- Permitir al empleado establecer su contraseña con ese token
- Login con emisión de token de acceso, y validación de dicho token
- Permitir el **cambio de contraseña de un usuario autenticado** (contraseña actual + nueva)
- Recuperación de contraseña por token
- **Activar y desactivar cuentas reaccionando a eventos**, distinguiendo la suspensión temporal (vacaciones) de la baja permanente (retiro)
- Almacenar las contraseñas **hasheadas** (BCrypt o equivalente)
- **Importante**: NO debe exponer un endpoint público de registro (`/register`). Las cuentas se crean **automáticamente de forma asíncrona** al consumir el evento `empleado.creado`. Solo el proceso interno event-driven puede crear cuentas, no usuarios externos.

4. Microservicio de Gestión de Perfiles

- Gestión del perfil personal del empleado
- Cada empleado autenticado puede actualizar **su propio** perfil, y solo el suyo: intentar modificar el perfil de otro empleado debe responder `403 Forbidden`
- Información complementaria: foto de perfil, biografía, redes sociales, dirección completa
- Sincronización con el microservicio de empleados vía eventos
- Archivado del perfil cuando el empleado es retirado

5. Microservicio de Gestión de Vacaciones

- CRUD de períodos de vacaciones
- Programación de vacaciones (fecha inicio, fecha fin)
- **Scheduler** que detecta el inicio y el fin de cada período y publica los eventos correspondientes, para que la cuenta se desactive y reactive automáticamente
- Validaciones: fechas coherentes, fechas no pasadas, no solapamiento de períodos, empleado existente

6. Microservicio de Notificaciones

- Envío de correos electrónicos
- Escuchar eventos del sistema (alta de usuario, vacaciones, activación y desactivación de cuenta)
- Plantillas de notificación configurables
- **Eventos a escuchar**:
  - `usuario.creado` → enviar bienvenida con el **token de activación** (nunca una contraseña)
  - `vacaciones.programadas` → enviar confirmación del período
  - `cuenta.desactivada` / `cuenta.activada` → notificar el cambio de estado de la cuenta

7. API Gateway

- Punto de entrada único para todas las peticiones del cliente
- Enrutamiento de peticiones a los microservicios correspondientes
- Validación de tokens (JWT propio o JWKS del IdP, según la opción elegida en el microservicio 3)
- **Nota**: No expone endpoint de registro público. Las cuentas se crean automáticamente vía eventos cuando RRHH registra un empleado
- **Operaciones expuestas**:

Método	Ruta	Descripción
`POST`	`/auth/login`	Autenticación
`POST`	`/auth/reset-password`	Establecer contraseña con el token de activación o recuperación
`POST`	`/auth/recover-password`	Solicitar recuperación de contraseña
`POST`	`/auth/change-password`	Cambio de contraseña (usuario autenticado)
`GET`	`/empleados`	Listar empleados. Admite `?estado=RETIRADO&desde=&hasta=` para auditoría
`POST`	`/empleados`	Crear empleado
`GET`	`/empleados/{id}`	Obtener un empleado
`PUT`	`/empleados/{id}`	Actualizar empleado
`DELETE`	`/empleados/{id}`	Retirar empleado (transición a `RETIRADO`, no borrado)
`GET`	`/departamentos`	Listar departamentos
`POST`	`/departamentos`	Crear departamento

Método	Ruta	Descripción
`GET`	`/perfiles/{empleadoId}`	Obtener perfil
`PUT`	`/perfiles/{empleadoId}`	Actualizar perfil (solo el propio)
`POST`	`/vacaciones`	Programar vacaciones
`GET`	`/vacaciones`	Consultar vacaciones

> **Note**
>
> **Composición de respuestas (opcional, otorga puntaje adicional).** Un endpoint agregado del tipo `GET /empleados/{id}/completo`, que combine en una sola respuesta los datos del empleado y
> los de su perfil, es un valor agregado del Gateway.
>
> Tenga en cuenta que **los gateways declarativos (Traefik, Kong) no componen respuestas**: son
> enrutadores. Para lograrlo necesita un Gateway con código propio (Express, Spring Cloud Gateway,
> FastAPI) o un BFF dedicado, resolviendo además los fallos parciales — qué responder si el servicio de perfiles no contesta. Es el patrón que se estudia a fondo en el Reto 27.

Microservicios de Observabilidad

8. Logs Centralizados

Implementar o integrar una solución de centralización de logs (ej: ELK Stack, Loki, Graylog)

- Todos los microservicios deben enviar logs estructurados
- Logs deben incluir: timestamp, nivel, servicio, mensaje, contexto

9. Monitoreo

Implementar o integrar una solución de monitoreo (ej: Prometheus + Grafana, Datadog)

- Métricas de salud de cada microservicio
- Métricas de negocio: empleados creados, notificaciones enviadas, errores
- Dashboards visuales

10. Trazabilidad Distribuida

Instrumentar los microservicios con **OpenTelemetry** y desplegar un servidor de trazas (Zipkin, Jaeger o equivalente)

- El `traceId` debe propagarse entre servicios de distintos lenguajes (estándar W3C Trace Context)
- Debe ser posible visualizar la **cascada completa** de una petición de onboarding: desde el Gateway hasta el último consumidor de eventos
- Los logs estructurados deben incluir el `traceId` para poder correlacionar log ↔ traza

Requisitos Técnicos por Microservicio

Cada microservicio que desarrolle debe cumplir con:

1. Documentación API

- Especificación OpenAPI (Swagger) completa
- Interfaz interactiva de prueba (Swagger UI)
- Documentación de modelos de datos y respuestas

2. Pruebas Automatizadas

- Pruebas unitarias de lógica de negocio
- Pruebas de integración que verifiquen el camino feliz de todas las operaciones
- Cobertura mínima: 70%

3. Health Check

- Implementar endpoint `GET /health` en todos los microservicios
- Responder con el estado del servicio y el de sus dependencias (base de datos, broker)
- Formato JSON:

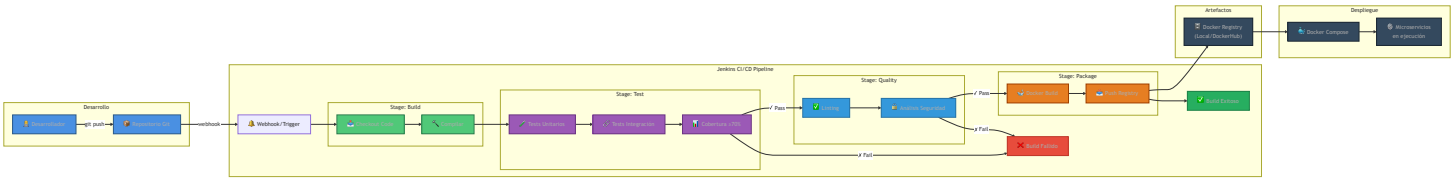
```
{
  "status": "UP",
  "service": "empleados-service",
  "checks": { "database": "UP", "messageBroker": "UP" }
}
```

- Retornar HTTP `200` cuando el estado es `UP` y HTTP `503` cuando es `DOWN`

4. Pipeline CI/CD

- Pipeline de integración continua en Jenkins
- Etapas mínimas: **build, test, calidad y package**
- Etapa de calidad con análisis estático y verificación de cobertura (SonarQube u equivalente); el pipeline debe **fallar** si la cobertura cae por debajo del 70 %
- Generación de artefactos (imágenes Docker)

Diagrama de Pipeline CI/CD



Flujo del Pipeline:

1. **Desarrollo**: El desarrollador hace push al repositorio Git
2. **Trigger**: Jenkins detecta cambios vía webhook
3. **Build**: Descarga código y compila el proyecto
4. **Test**: Ejecuta tests unitarios e integración, valida cobertura ≥70%
5. **Quality**: Análisis de código (linting) y seguridad
6. **Package**: Construye imagen Docker y la publica en Registry
7. **Deploy**: Docker Compose usa las imágenes para desplegar servicios

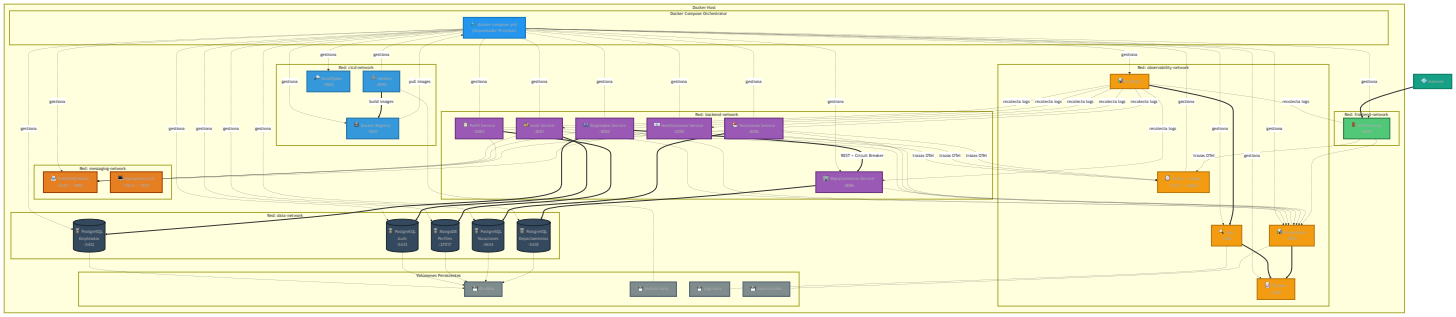
5. Contenerización

- Dockerfile para cada microservicio
- Imágenes optimizadas (multi-stage builds recomendado)
- Variables de entorno para configuración

6. Orquestación

- Docker Compose que despliegue todo el sistema con un solo comando
- Incluir todos los microservicios y dependencias (bases de datos, message broker, etc.)
- Incluir una instancia de Jenkins para ejecutar los pipelines

Diagrama de Infraestructura y Despliegue



Componentes de la Infraestructura:

- **Docker Compose**: Orquestador que levanta todos los servicios con `docker compose up`
- **Redes Docker**: Segmentación por capas (frontend, backend, messaging, data, observability, CI/CD)
- **Volúmenes Persistentes**: Almacenamiento permanente para bases de datos, Jenkins, logs y métricas
- **Servicios Core**: 6 microservicios de negocio + API Gateway
- **Message Broker**: RabbitMQ o Kafka para comunicación asíncrona
- **Bases de Datos**: una por microservicio (PostgreSQL, MongoDB o la que corresponda)
- **Observabilidad**: Loki + Promtail (logs) + Prometheus + Grafana (métricas y alertas) + Zipkin/Jaeger (trazas)
- **CI/CD**: Jenkins + SonarQube + Docker Registry local

> **Sobre el stack de observabilidad:** el diagrama muestra Loki + Promtail porque es el stack que se construye en el Reto 8 y se integra nativamente con Grafana. Puede sustituirlo por ELK (Elasticsearch + Logstash + Kibana), Graylog o una solución gestionada, **siempre que cubra los mismos requisitos funcionales y lo justifique en el README**.

> **Sobre los puertos:** los puertos del diagrama son una referencia. Verifique que no haya colisiones en el host — es un error común publicar dos servicios en el mismo puerto (por ejemplo, Jenkins y Prometheus en `:9090`).

Comando de inicio:

```
docker compose up --build
```

Arquitectura del Sistema

Comunicación entre Microservicios

Los microservicios deben comunicarse mediante **mensajería asíncrona** utilizando un message broker de su elección (RabbitMQ, Apache Kafka, Redis Streams, NATS, etc.).

> **Important**
>
> - **Comunicación sincrónica (REST)**: Solo entre Cliente ↔ API Gateway ↔ Microservicios
> - **Comunicación asíncrona (Events)**: Entre microservicios a través del message broker
> - **NO exponer endpoints públicos de registro**: Las credenciales se crean automáticamente vía eventos internos

Eventos Principales

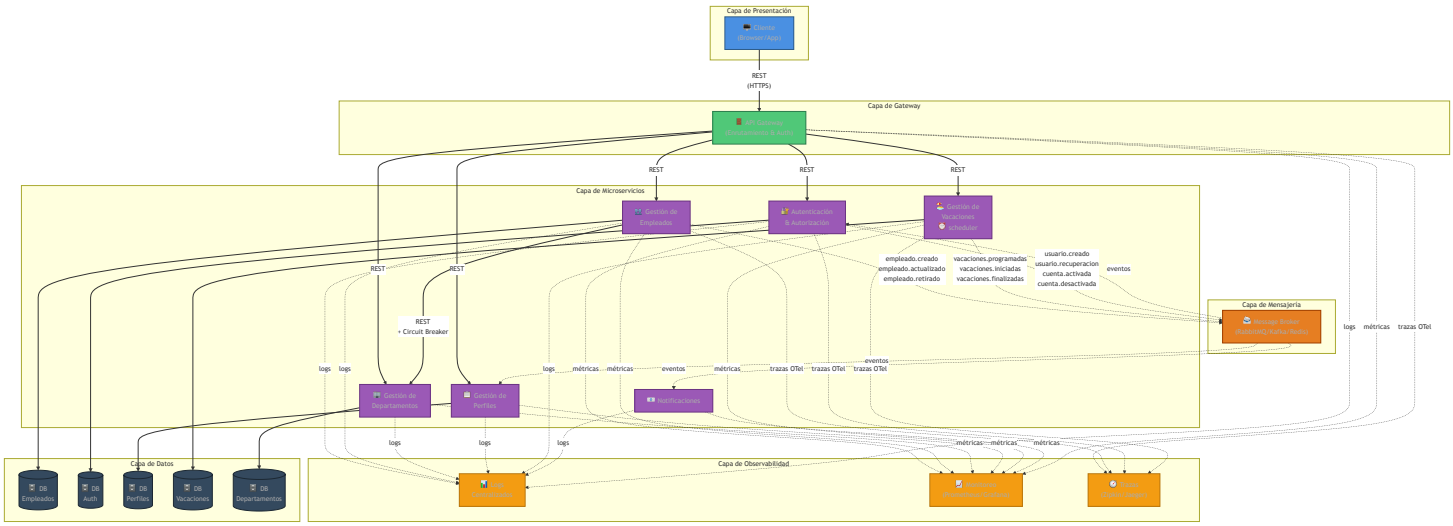
> **Important**
>
> La definición completa y normativa de los eventos —nombres, envelope, cargas útiles, productores
> y consumidores— está en el **Catálogo de Eventos**. La tabla siguiente
> es solo un resumen. **En caso de discrepancia, prevalece el catálogo.**

Evento	Publicado por	Consumido por
`empleado.creado`	Gestión de Empleados	Autenticación, Perfiles, Notificaciones
`empleado.actualizado`	Gestión de Empleados	Perfiles
`empleado.retirado`	Gestión de Empleados	Autenticación, Perfiles, Notificaciones
`usuario.creado`	Autenticación	Notificaciones
`usuario.recuperacion`	Autenticación	Notificaciones
`cuenta.activada`	Autenticación	Notificaciones
`cuenta.desactivada`	Autenticación	Notificaciones
`vacaciones.programadas`	Gestión de Vacaciones	Notificaciones

Evento	Publicado por	Consumido por
`vacaciones.iniciadas`	Gestión de Vacaciones	Autenticación, Notificaciones
`vacaciones.finalizadas`	Gestión de Vacaciones	Autenticación, Notificaciones

Todos los eventos viajan en un envelope común con `id`, `type`, `version`, `occurredAt`, `producer` y `data`. El campo `id` existe para que **los consumidores puedan descartar mensajes duplicados**; implementar esa deduplicación es obligatorio.

Diagrama de Arquitectura (Referencia)



Leyenda:

- **Líneas sólidas (→)**: Comunicación sincrónica REST/HTTPS
- **Líneas punteadas (→)**: Comunicación asíncrona mediante eventos
- **Emojis**: Identificación visual rápida de cada componente

Restricciones Técnicas

> **Caution**
>
> El incumplimiento de estas restricciones afectará directamente su calificación.

Restricciones Obligatorias

1. **Comunicación Asíncrona**
   - Los microservicios DEBEN comunicarse mediante un sistema de gestión de mensajes (message broker)
   - Comunicación sincrónica (REST) solo se permite entre API Gateway y microservicios
2. **Diversidad Tecnológica**
   - Debe emplear **al menos 4 lenguajes de programación diferentes** para el desarrollo de los microservicios
   - Ejemplos: Java, Python, Node.js, Go, C#, Ruby, etc.
3. **Despliegue Unificado**
   - Debe existir un archivo `docker-compose.yml` en la raíz del proyecto
   - Con **un solo comando** (`docker compose up --build`) todo el sistema debe:
     - Compilarse
     - Empaquetarse
     - Desplegarse
   - Incluir una instancia de Jenkins en el Docker Compose
4. **Cero Secretos en el Repositorio**
   - Ninguna contraseña, token, clave de firma o API key puede estar en el código fuente, en el `docker-compose.yml` ni en el historial de Git
   - El repositorio debe contener un `.env.example` con marcadores de posición, y un `.gitignore` que excluya `.env` y `secrets/`
   - Debe incluirse un script de inicialización que genere los secretos localmente
   - **Criterio de verificación:** el repositorio debería poder publicarse como código abierto en cualquier momento sin comprometer ninguna credencial
5. **Entregables**
   - Código fuente comprimido (.zip o .tar.gz)
   - Al descomprimir debe quedar un directorio con estructura clara
   - Incluir archivo README.md con instrucciones de ejecución

Criterios de Evaluación

La evaluación del proyecto se realizará considerando los siguientes aspectos:

#	Criterio	Peso
1	Funcionalidad	35 %
2	Calidad Técnica	25 %
3	Observabilidad	15 %
4	Seguridad y Configuración	10 %
5	Escalabilidad y Resiliencia	10 %
6	DevOps	5 %
7	Documentación	5 %

1. Funcionalidad (35%)

- Todos los microservicios implementados y funcionando
- Flujos de negocio completos: onboarding, gestión de perfil, vacaciones, offboarding
- **Ciclo de vida de la cuenta funcionando de extremo a extremo**: activación, suspensión automática al iniciar vacaciones, reactivación al terminar, baja permanente al retirar
- Comunicación mediante eventos funcionando correctamente, respetando el Catálogo de Eventos
- Consumidores tolerantes a eventos duplicados
- API Gateway enrutando correctamente las peticiones

2. Calidad Técnica (25%)

- Especificaciones OpenAPI completas y correctas
- Pruebas automatizadas implementadas y pasando (unitarias, integración y funcionales)
- Cobertura mínima del 70 %
- Health checks implementados
- Dockerfiles correctos y optimizados
- Código limpio y bien estructurado

3. Observabilidad (15%)

- Logs centralizados y estructurados funcionando
- Sistema de monitoreo implementado
- Dashboards con métricas relevantes
- Alertas configuradas hacia un canal externo

- **Trazabilidad distribuida**: traza completa de extremo a extremo del flujo de onboarding, visible en Zipkin/Jaeger, con `traceId` correlacionado en los logs

4. Seguridad y Configuración (10%)

- **Ningún secreto en el repositorio**: sin contraseñas, tokens ni claves en el código, en el `docker-compose.yml` ni en el historial de Git
- `.env.example` completo y versionado; `.env` y `secrets/` excluidos por `.gitignore`
- Secretos gestionados con Docker Compose Secrets (o Vault)
- Contraseñas hasheadas; ninguna contraseña transmitida por correo ni por eventos
- Control de acceso correcto: RBAC por rol **y** autorización por propiedad del recurso
- *(Puntaje adicional)* Identity Provider con OAuth2/OIDC y validación por JWKS

5. Escalabilidad y Resiliencia (10%)

- Servicios escalables horizontalmente y **stateless**
- Load balancer distribuyendo el tráfico entre instancias, con health checks
- Al menos un **Circuit Breaker** con fallback demostrable en la comunicación sincrónica
- Manejo correcto de timeouts y reintentos
- El sistema sigue atendiendo peticiones cuando cae una instancia

6. DevOps (5%)

- Pipelines de CI en Jenkins funcionando
- Docker Compose desplegando todo el sistema correctamente
- Documentación de despliegue clara

7. Documentación (5%)

- README con instrucciones claras
- Arquitectura documentada
- Guías de configuración de herramientas de terceros

> **Note**
>
> **Correspondencia con los talleres.** Cada criterio se construye en los retos del semestre: Funcionalidad (Retos 1–5), Calidad Técnica (Retos 2, 6, 7), Observabilidad (Reto 8), Seguridad y Configuración (Retos 5, 9, 10), Escalabilidad y Resiliencia (Retos 3, 11),

DevOps (Reto 7). Un equipo que resuelve los talleres tiene el proyecto prácticamente construido.

Entregables

Debe entregar un archivo comprimido que contenga:

Estructura del Proyecto

```
proyecto-final-microservicios/
│
├── README.md                          # Instrucciones generales
├── docker-compose.yml                 # Orquestación completa
├── docker-compose.dev.yml             # (Opcional) Configuración para desarrollo
│
├── .env.example                       # Plantilla de variables (el .env real NO se ver
├── .gitignore                         # Debe excluir .env y secrets/
├── secrets.example/                   # Plantillas de los archivos de secretos
│
├── microservicios/
│   ├── gestion-empleados/             # Microservicio 1
│   │   ├── Dockerfile
│   │   ├── Jenkinsfile
│   │   ├── src/
│   │   ├── tests/
│   │   └── openapi.yaml
│   │
│   ├── gestion-departamentos/         # Microservicio 2
│   │   ├── Dockerfile
│   │   ├── Jenkinsfile
│   │   ├── src/
│   │   ├── tests/
│   │   └── openapi.yaml
│   │
│   ├── autenticacion/                 # Microservicio 3
│   ├── gestion-perfiles/              # Microservicio 4
│   ├── gestion-vacaciones/            # Microservicio 5
│   ├── notificaciones/                # Microservicio 6
│   └── api-gateway/                   # Microservicio 7
│
├── e2e-tests/                         # Suite de pruebas funcionales (BDD)
│
├── observabilidad/
│   ├── logs/                          # Configuración de logs centralizados
│   ├── monitoreo/                     # Configuración de monitoreo
│   ├── trazas/                        # Configuración del servidor de trazas
│   └── dashboards/                    # Dashboards de Grafana o similar
│
├── docs/
│   ├── arquitectura.md                # Diagrama y descripción de arquitectura
```

```
│   ├── eventos.md                    # Eventos implementados (según el catálogo del c
│   └── guias/                        # Guías de configuración
│
└── scripts/
    ├── setup-secrets.sh              # Generación de secretos locales
    ├── init-db.sql                   # Scripts de inicialización
    └── seed-data.sql                 # Datos de prueba
```

README.md Mínimo

El archivo README.md debe incluir:

1. **Descripción del Proyecto**
   - Resumen de la solución implementada
   - Arquitectura general
2. **Requisitos Previos**
   - Docker y Docker Compose instalados
   - Versiones mínimas requeridas
3. **Instrucciones de Ejecución**

```
# Clonar o descomprimir el proyecto
cd proyecto-final-microservicios

# Generar la configuración local (una sola vez)
cp .env.example .env
bash scripts/setup-secrets.sh

# Levantar todo el sistema
docker compose up --build

# Acceder a los servicios
# - API Gateway:  http://localhost:8080
# - Swagger UI:   http://localhost:8080/swagger
# - Jenkins:      http://localhost:8090
# - Grafana:      http://localhost:3000
# - Prometheus:   http://localhost:9090
# - Trazas:       http://localhost:9411   (Zipkin) o :16686 (Jaeger)
```

4. **Tecnologías Utilizadas**
   - Tabla **microservicio ↔ lenguaje de programación** (mínimo 4 lenguajes distintos)
   - Message broker utilizado

- Bases de datos utilizadas
- Herramientas de observabilidad

5. **Endpoints Principales**
   - Listado de endpoints del API Gateway
   - Ejemplos de uso con curl
6. **Configuración Inicial y Secretos**
   - Cómo generar el `.env` y los archivos de `secrets/`
   - Qué variable configura cada cosa
   - Procedimiento de rotación de secretos
7. **Accesos por Defecto**
   - Cómo se crea el usuario administrador semilla
   - Accesos a las herramientas (Grafana, Jenkins, broker, etc.)
   - **Sin contraseñas reales en el documento**: indique de qué variable de entorno o archivo de secreto proviene cada credencial

Recomendaciones

Gestión del Proyecto

1. **Planificación**
   - Dividan el trabajo en sprints
   - Definan interfaces de eventos primero
   - Implementen el message broker temprano
2. **Desarrollo Incremental**
   - Comiencen con 2-3 microservicios básicos
   - Agreguen observabilidad rápidamente
   - Integren continuamente
3. **Aprovechen los talleres**
   - El proyecto **no es un trabajo aparte**: es el mismo sistema que se construye reto a reto. Un equipo que resuelve los talleres del semestre llega a la entrega con el proyecto casi terminado.
   - Mantengan un solo repositorio (o conjunto de repositorios) que evolucione con cada reto, en lugar de empezar de cero para la entrega final.
4. **Pruebas**
   - Prueben cada microservicio de manera aislada
   - Prueben la integración completa frecuentemente

- Automaticen las pruebas desde el inicio

Herramientas Sugeridas

Componente	Opciones Sugeridas
Message Broker	RabbitMQ, Apache Kafka, Redis Streams, NATS
Logs Centralizados	Loki + Promtail, ELK Stack (Elasticsearch + Logstash + Kibana), Graylog
Monitoreo	Prometheus + Grafana
Trazabilidad	OpenTelemetry + Zipkin o Jaeger
Identity Provider	Keycloak, Authentik (opción **(b)** del microservicio 3)
API Gateway	Traefik, Kong, Nginx, Spring Cloud Gateway, Express Gateway, propio
Gestión de Secretos	Docker Compose Secrets, HashiCorp Vault
Calidad de Código	SonarQube
Base de Datos	PostgreSQL, MySQL, MongoDB (según microservicio)

Lenguajes Sugeridos

- **Java/Spring Boot**: Microservicios empresariales (empleados, autenticación)
- **Python/FastAPI**: Microservicios de datos y procesamiento (notificaciones)
- **Node.js/Express**: API Gateway
- **Go**: Servicios de alto rendimiento (logs, monitoreo)

Soporte y Consultas

Para aclaraciones sobre el proyecto:

1. Revise este documento completamente
2. Consulte la documentación de las tecnologías elegidas
3. Las herramientas de terceros deben incluir guía de configuración
4. Asista a las sesiones de asesoría del curso

Equipos de trabajo

El proyecto está pensado para que sea desarrollado en grupos de máximo 4 personas.

Fecha de Entrega

Semana 15

La entrega se realizará mediante **Class Room**.

Recursos Adicionales

- Documentación de Docker Compose
- Especificación OpenAPI
- Patrones de Microservicios
- Event-Driven Architecture
- The Twelve-Factor App

> **IMPORTANTE**
> Este proyecto es una oportunidad para aplicar todos los conocimientos adquiridos en el curso. Se espera profesionalismo, calidad técnica y trabajo en equipo. Buena suerte!